SENAI CIMATEC
Programa QuIIN

Problem-Based Learning 03 (PBL03)

Processador RISC-V de ISA Limitada em SystemVerilog

Desenvolvimento, validação e execução do jogo Genius
Relatório da Entrega 3 — Frente 1

Marina de Oliveira Alencar

Outubro de 2026

Resumo

Este relatório apresenta a Frente 1 do PBL03: o desenvolvimento de um processador RISC-V de ciclo único com conjunto de instruções limitado, descrito em SystemVerilog e baseado na microarquitetura do Capítulo 7 de Harris e Harris. O processador implementa 22 instruções, das quais seis são extensões além do que o capítulo apresenta, necessárias para executar o jogo Genius desenvolvido em assembly no PBL02. A validação foi feita no Vivado Simulator 2024.1: um programa de sanidade conferiu os registradores no `ecall` e o Genius foi executado sem nenhuma alteração no código em cinco cenários, que cobrem os modos "Siga" e "Mando eu", três dificuldades e duas velocidades. Em todos os cenários a pontuação final e a sequência de cores coincidiram com o esperado (`5 cenarios passaram, 0 falharam`). Os diagramas gerados pelo Vivado mostram a divisão em controller e datapath do livro. A verificação em UVM do processador da outra equipe (Frente 2) ainda não foi iniciada.

Palavras-chave: RISC-V; SystemVerilog; processador de ciclo único; Vivado; Genius.

Sumário

1 Introdução **1**
1.1 Contexto e objetivo 1
1.2 Escopo 1
1.3 Organização do documento 1

2 Referência: o que do Capítulo 7 foi seguido **2**
2.1 Extensões além do capítulo 3

3 Arquitetura implementada **3**

4 Passo a passo do desenvolvimento **4**

5 Validação com o testbench de sanidade **6**

6 Genius rodando no processador **7**
6.1 Montagem do sistema 7
6.2 Testbench e cenários 7
6.3 Resultados no Vivado 8

7 Diagramas gerados pelo Vivado **11**
7.1 Como os diagramas foram obtidos 12

8 Conclusão e próximos passos **12**
8.1 Limitações 12
8.2 Próximos passos 13

A Como reproduzir **13**

B Código do SoC de teste do Genius **15**

Lista de Figuras

1 Módulo `riscvsingle` no Vivado (*elaborated design*). O controller (`c`) recebe `op`, `funct3`, `funct7b5` e as flags `zero`, `lt` e `ltu`; o datapath (`dp`) devolve `PC`, `ALUResult` e `WriteData`. 4
2 Ondas do teste de sanidade no Vivado Simulator (`xsim -gui`): `clk`, `reset`, `WriteData`, `DataAdr`, `MemWrite`, `ECall`, `PC` e `Instr`. 6
3 Console do `xsim`: valores dos registradores no `ecall`, todos iguais aos esperados. 6
4 Execução no `xsim`: cada cenário termina com `PASSOU`. 8
5 Console do Vivado: cenário 5 (Mando eu) com pontuação 3 e o resultado final `5 cenarios passaram, 0 falharam`. 9
6 Ondas do Vivado na simulação inteira: `PC`, `led_prev`, `dpad_btn`, `score` e `ecall`. 9
7 Modo Siga (cenário 2): a sequência de LEDs cresce a cada rodada. . . . 10
8 Modo Mando eu (cenário 5): apertos de botão e sequência repetida nos LEDs. 10
9 O `PC` muda a cada ciclo de clock na escala de nanossegundos (próximo de 895 µs). 11
10 `genius_soc`: processador (`rv`), memórias (`imem1`, `dmem1`) e periféricos do Genius (`pp`). 11
11 O módulo `rv` (`riscvsingle`) aberto dentro do SoC. 12

Lista de Tabelas

1 Correspondência entre o Capítulo 7 e o projeto. 2
2 Extensões ao processador do Capítulo 7. 3
3 Arquivos do RTL do processador. 4
4 ISA implementada (22 instruções). 5
5 Resultado do teste de sanidade no `ecall`. 6
6 Mapa de memória do SoC de teste. 7
7 Cenários simulados e resultados. 8

1 Introdução

1.1 Contexto e objetivo

O PBL03 dá continuidade aos dois projetos anteriores da trilha: no PBL01, o jogo Genius foi implementado como máquina de estados em hardware (RTL); no PBL02, o mesmo jogo foi reescrito em assembly RISC-V e validado no simulador Ripes. No PBL03, o jogo passa a rodar em um processador projetado pela própria equipe, e o processador de outra equipe é verificado com a metodologia UVM. O trabalho segue a estrutura de projeto do repositório `verif-env-template` [2], com Vivado como ferramenta de simulação.

Este relatório cobre a **Frente 1**, o desenvolvimento do processador. O objetivo é construir, em SystemVerilog, um processador RISC-V de ciclo único com ISA limitada, seguindo a microarquitetura do Capítulo 7 de Harris e Harris [1], e demonstrar que ele executa o programa Genius do PBL02 sem alterações no código.

1.2 Escopo

O que o relatório cobre:

- o processador de ciclo único (controller e datapath), com 22 instruções;
- a comparação, ponto a ponto, com o que o Capítulo 7 apresenta, e a justificativa de cada extensão;
- a validação com um programa de sanidade;
- a execução do Genius em simulação, com periféricos modelados em RTL;
- os diagramas gerados pelo Vivado e o passo a passo para reproduzir tudo.

O que ficou fora, e está registrado na Seção 8: a Frente 2 (ambiente UVM e verificação do processador da outra equipe), a síntese e o teste em placa.

1.3 Organização do documento

A Seção 2 compara o projeto com o Capítulo 7. A Seção 3 descreve a arquitetura e os arquivos do RTL. A Seção 4 registra o passo a passo do desenvolvimento e a ISA implementada. A Seção 5 mostra a validação básica, e a Seção 6 a execução do Genius. A Seção 7 apresenta os diagramas do Vivado. A Seção 8 traz a conclusão, as limitações e os próximos passos. Os Apêndices A e B reúnem os comandos de reprodução e o código do SoC de teste.

2 Referência: o que do Capítulo 7 foi seguido

A base do projeto é o processador de ciclo único do Capítulo 7 de *Digital Design and Computer Architecture: RISC-V Edition* [1]. O capítulo constrói o datapath e o controle para `lw`, `sw`, as instruções tipo R (`add`, `sub`, `and`, `or`) e `beq`, e depois o estende com `addi` e `jal`. A organização em `riscvsingle` (controller e datapath), com memórias de instrução e de dados separadas, foi mantida. O que o programa Genius exigiu além disso foi acrescentado como extensão. A Tabela 1 mostra, elemento por elemento, o que foi seguido e o que foi estendido.

Tabela 1: Correspondência entre o Capítulo 7 e o projeto.

Elemento do Capítulo 7	**No projeto**	**Situação**
Ciclo único: uma instrução por ciclo, sem estado intermediário	`riscvsingle` = `controller` + `datapath`	Seguido
Banco de 32 registradores, duas leituras e uma escrita, `x0` fixo em 0	`regfile.sv`	Seguido
Unidade de extensão do imediato (formatos I, S, B, J)	`extend.sv`	Seguido; ganhou o formato U (`lui`/`auipc`)
ALU com `ALUControl`	`alu.sv`	Seguido; ganhou as flags `lt` (com sinal) e `ltu` (sem sinal)
Main Decoder e ALU Decoder em dois níveis, com `ALUOp`	`maindec.sv`, `aludec.sv`	Seguido; ganhou `ALUSrcA`, `Jalr` e `ecall`
`PCSrc`: PC+4 ou alvo de desvio	`controller.sv`	Estendido: mux de 3 vias, incluindo `ALUResult` (`jalr`)
`ResultSrc`: ALU, memória ou PC+4 (`jal`)	`datapath.sv`	Estendido: 4 vias, incluindo o imediato (`lui`)
Memória de dados com acesso por palavra (`lw`/`sw`)	`dmem.sv`	Estendido: acesso a byte (`lb`/`sb`) por `funct3`
`beq`, `jal` e `addi` do capítulo	Todos implementados	Seguido

2.1 Extensões além do capítulo

O assembly do Genius usa instruções que o livro não cobre, o que levou às seis extensões da Tabela 2. A ISA completa segue o formato definido pela especificação RISC-V [3].

Tabela 2: Extensões ao processador do Capítulo 7.

Extensão	Por quê	O que mudou no hardware
`lb` / `sb`	O jogo acessa vetores de bytes (sequência e cores)	`dmem.sv` seleciona a largura por `funct3` e faz extensão de sinal no `lb`
`bne` / `bge` / `bltu`	O livro só ensina `beq`; o jogo precisa de diferença, maior ou igual (com sinal) e menor (sem sinal)	Flags `lt` e `ltu` na ALU; `controller.sv` combina `funct3` com as flags em `branch_taken`
`jalr`	Retorno de sub-rotina por endereço em registrador	`PCSrc` passa a 3 vias, incluindo o `ALUResult`
`lui`	O imediato de 12 bits do `addi` não alcança os endereços do jogo	Formato U no `extend.sv`; `ResultSrc` com 4 opções
`auipc`	Constrói endereços relativos ao PC (usado em `la`)	Mux `ALUSrcA` (registrador ou PC) na entrada da ALU
`ecall`	Ponto de parada controlado, para o testbench encerrar a simulação	Sinal `ECall` dedicado, levantado pelo controller

3 Arquitetura implementada

Como no Capítulo 7, o processador é o módulo `riscvsingle`, formado por duas partes: o **controller**, que decodifica a instrução e gera os sinais de controle, e o **datapath**, que executa a instrução. As memórias ficam fora, no módulo de topo `riscv_single_top`.

O fluxo de uma instrução segue o livro. O `PC` endereça a memória de instruções e a instrução chega ao controller (`op`, `funct3`, `funct7b5`) e ao datapath. O banco de registradores lê `rs1` e `rs2`, a unidade de extensão gera o imediato conforme `ImmSrc`, e a ALU calcula com a entrada B escolhida por `ALUSrcB` (e, no `auipc`, a entrada A por `ALUSrcA`). A memória de dados é lida ou escrita por `lw`/`lb` e `sw`/`sb`. O resultado volta ao banco de registradores, escolhido por `ResultSrc` (ALU, memória, PC+4 ou imediato), e o próximo `PC` vem de `PCSrc`: PC+4, alvo de desvio ou `ALUResult` no `jalr`.

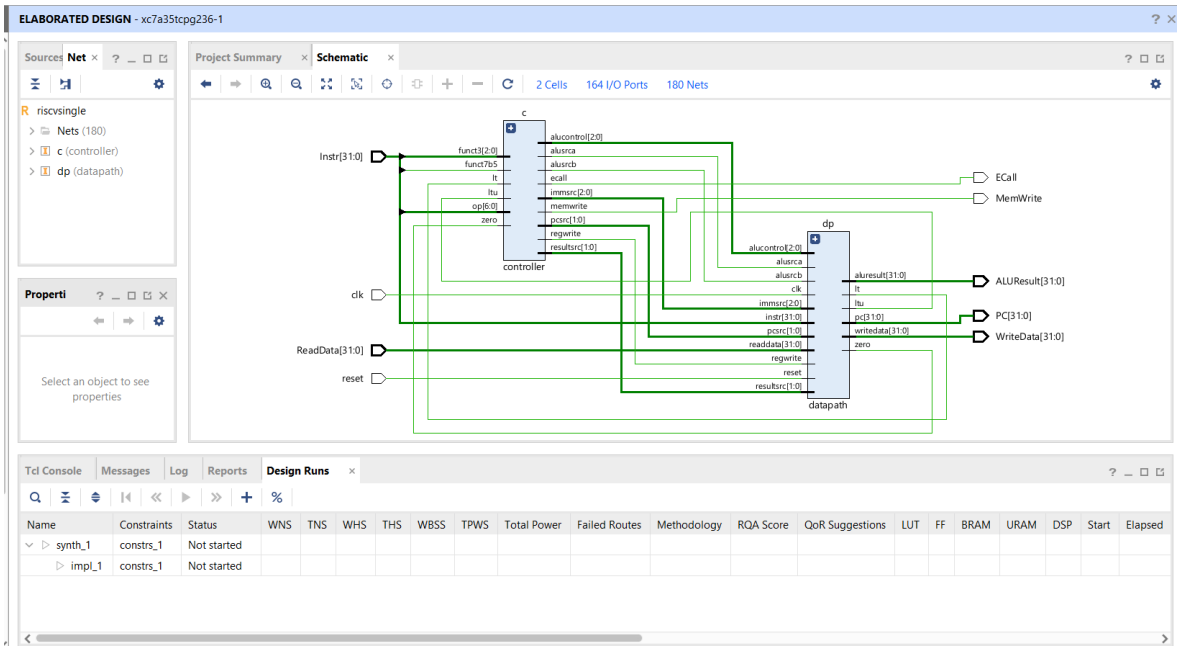


Figura 1: Módulo `riscvsingle` no Vivado (*elaborated design*). O controller (`c`) recebe `op`, `funct3`, `funct7b5` e as flags `zero`, `lt` e `ltu`; o datapath (`dp`) devolve `PC`, `ALUResult` e `WriteData`.

A Tabela 3 lista os arquivos do RTL e a função de cada um.

Tabela 3: Arquivos do RTL do processador.

Arquivo	Função
`rtl/riscvsingle.sv`	Une controller e datapath
`rtl/controller.sv`	Liga o Main Decoder e o ALU Decoder; decide `PCSrc` e `branch_taken`
`rtl/maindec.sv`	Gera os sinais de controle a partir do `opcode`
`rtl/aludec.sv`	Gera `ALUControl` a partir de `ALUOp`, `funct3` e `funct7b5`
`rtl/datapath.sv`	PC, banco de registradores, extensão, ALU e multiplexadores
`rtl/alu.sv`	Operações aritméticas e lógicas; flags `zero`, `lt`, `ltu`
`rtl/regfile.sv`	32 registradores de 32 bits
`rtl/extend.sv`	Imediatos dos formatos I, S, B, J e U
`rtl/imem.sv`, `rtl/dmem.sv`	Memórias de instrução e de dados (byte e palavra)
`rtl/riscv_single_top.sv`	Processador mais `imem` e `dmem`, usado no teste de sanidade

4 Passo a passo do desenvolvimento

A ordem abaixo é a que foi seguida, do estudo do Capítulo 7 até o jogo rodando.

1. **Levantar as instruções necessárias.** Com o assembly do Genius (PBL02) em mãos, foram listadas todas as instruções usadas. Parte delas já está no Capítulo 7; o resto virou extensão (Seção 2).
2. **Definir a ISA limitada.** Ficaram 22 instruções, listadas na Tabela 4.
3. **Implementar o núcleo do Capítulo 7.** Banco de registradores, extensão do imediato, ALU, Main Decoder, ALU Decoder, controller e datapath, nesta ordem, cada um como módulo separado.
4. **Acrescentar as extensões.** `lb`/`sb` na memória de dados; flags `lt`/`ltu` para `bne`/`bge`/`bltu`; `PCSrc` de 3 vias para `jalr`; formato U e `ResultSrc` de 4 vias para `lui`; `ALUSrcA` para `auipc`; sinal `ECall`.
5. **Montar o topo e as memórias.** `riscv_single_top` liga o processador a `imem` e `dmem`; o programa é carregado por `$readmemh`.
6. **Escrever o teste de sanidade.** Um programa curto (`sim/encode.py` gera o `program.hex`) exercita as classes de instrução, e `tb/testbench.sv` confere os registradores no `ecall` (Seção 5).
7. **Validar no fluxo oficial.** Checagem rápida com Icarus Verilog e depois execução no Vivado (`xvlog`, `xelab`, `xsim`) pelo `verif-env-template`, com CMake [2, 4].
8. **Rodar o Genius.** Um SoC de teste liga o núcleo a periféricos mapeados em memória, e um testbench joga o jogo e confere o resultado (Seção 6).
9. **Gerar os diagramas.** *Elaborated design* do Vivado, para mostrar que o RTL corresponde ao desenho do capítulo (Seção 7).

Tabela 4: ISA implementada (22 instruções).

Instrução	Tipo	Opcode	Função
`add`, `sub`, `and`, `or`, `xor`	R	`0110011`	Operações entre registradores
`addi`, `andi`, `slli`, `srli`	I	`0010011`	Operações com imediato
`lw`, `lb`	I	`0000011`	Carrega palavra / byte (com extensão de sinal)
`sw`, `sb`	S	`0100011`	Armazena palavra / byte
`beq`, `bne`, `bge`, `bltu`	B	`1100011`	Desvios condicionais
`jal`	J	`1101111`	Salto com retorno
`jalr`	I	`1100111`	Salto indireto com retorno
`lui`	U	`0110111`	Imediato nos bits altos
`auipc`	U	`0010111`	PC mais imediato nos bits altos
`ecall`	I	`1110011`	Marcador de fim da simulação

5 Validação com o testbench de sanidade

Antes do Genius, um programa curto exercita uma instrução de cada classe: aritmética, `sw`/`lw`, `sb`/`lb`, um laço com `bltu`, `auipc` e o `ecall` final. O `tb/testbench.sv` instancia `riscv_single_top`, gera o clock e, ao ver o `ecall`, confere os registradores. A Tabela 5 mostra o resultado.

Tabela 5: Resultado do teste de sanidade no `ecall`.

Registrador	Papel no teste	Esperado	Obtido
`x28` (`t3`)	lido de volta da memória (`sw` + `lw`)	15	15
`x30` (`t5`)	lido de volta da memória (`sb` + `lb`)	65	65
`x12` (`a2`)	`auipc` no endereço `0x34`	`0x34`	`0x34`
`x10` (`a0`)	contador do laço (`bltu`)	3	3

A simulação termina no `$finish` em 205 ns. A Figura 2 mostra as ondas do Vivado Simulator para o programa inteiro, e a Figura 3 o console com os valores finais.

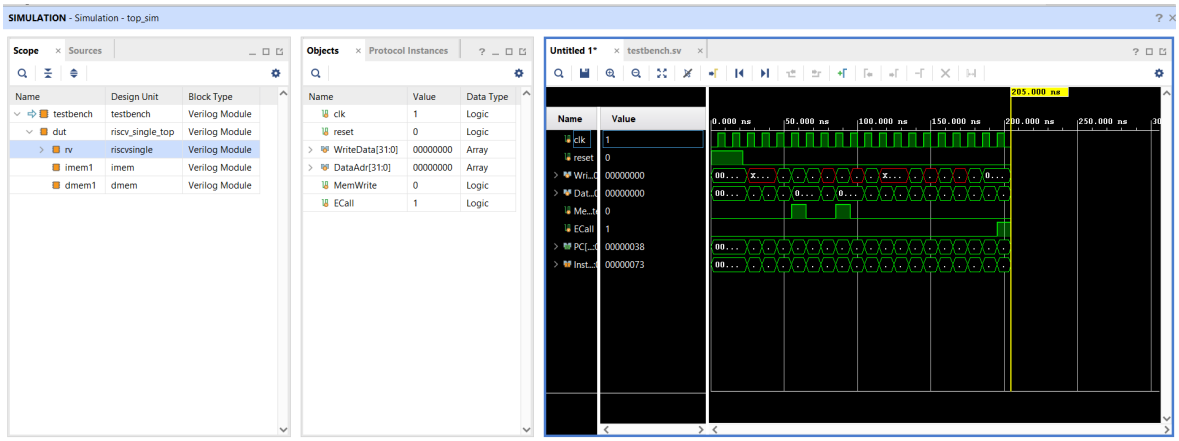


Figura 2: Ondas do teste de sanidade no Vivado Simulator (`xsim -gui`): `clk`, `reset`, `WriteData`, `DataAdr`, `MemWrite`, `ECall`, `PC` e `Instr`.

```
---- ECALL em PC=00000038 ----
a0 = 3 (0x00000003)
t3 (mem[0x2000]) esperado=15 -> x28 = 15
t5 (mem[0x2004]) esperado=65 -> x30 = 65
a2 (auipc)       esperado=0x34 -> x12 = 00000034
$finish called at time : 205 ns : File "C:/Users/marin/Documents/pbl03/tb/testbench.sv" Line 44
```

Figura 3: Console do `xsim`: valores dos registradores no `ecall`, todos iguais aos esperados.

A validação foi feita em dois níveis: uma checagem rápida de sintaxe e funcionalidade com Icarus Verilog durante o desenvolvimento, e a execução no fluxo oficial da disciplina, com o Vivado pelo `verif-env-template` (CMake com `xvlog`, `xelab` e

`xsim`) [2, 4]. Na máquina utilizada (Windows com Git Bash), o `xsim` precisa ser chamado por `cmd //c`, porque o script de inicialização do Xilinx reclama de `Unsupported architecture: i686-pc` no Git Bash; o procedimento completo está no Apêndice A.

6 Genius rodando no processador

O programa Genius do PBL02, em assembly RISC-V, roda no processador **sem nenhuma alteração no código**. A verificação é feita somente em simulação: os periféricos do jogo são modelos em RTL, e não hardware da Basys3.

6.1 Montagem do sistema

O jogo precisa de LEDs, chaves, botões e um relógio, que o núcleo não tem. Um SoC de teste (`genius_soc.sv`) liga o núcleo às memórias e a um bloco de periféricos mapeados em memória (`genius_periph.sv`); o código está no Apêndice B. Os endereços dos periféricos entram na montagem como símbolos (`-defsym`), e um pequeno `crt0.s` inicializa o `sp`, como o Ripes faz. O programa é montado com `riscv64-unknown-elf-as -march=rv32i`, ligado com `.text` em `0x0` e `.data` em `0x2000`, e convertido em `genius.hex` (161 palavras) e `data.hex` (48 bytes). A Tabela 6 mostra o mapa de memória.

Tabela 6: Mapa de memória do SoC de teste.

Endereço	O que é	Acesso
`0x0000_0000`	Memória de instruções (`genius.hex`)	leitura
`0x0000_2000`	Dados: `seq_buffer` em `0x2000`, `cores` em `0x2020` (`data.hex`)	lb/sb
`0x0000_3FF0`	Topo da pilha (`sp`, definido pelo `crt0.s`)	lw/sw
`0xF000_0000`	LEDs: 4 LEDs de 24 bits (UP, DOWN, LEFT, RIGHT)	escrita
`0xF000_0100`	Chaves: bit 4 início, bit 3 modo, bit 2 velocidade, bits 1:0 dificuldade	leitura
`0xF000_0200`	D-Pad: 4 botões com trava, limpa quando o programa lê	leitura
`0xF000_0300`	Relógio (RTC): avança 1 ms por ciclo de clock	leitura

O desassembly confirmou que todas as instruções do Genius já estavam na ISA implementada, então o núcleo não precisou de nenhuma mudança para o jogo.

6.2 Testbench e cenários

O `tb/genius_tb.sv` joga o jogo sozinho: aperta os botões do D-Pad, observa os LEDs e compara cada cor mostrada com um modelo de referência (LFSR de 8 bits, semente

`0xA5`, *taps* nos bits 7, 5, 4 e 3). Também confere a pontuação impressa no `ecall` e o maior tamanho de sequência (registrador `s3`). A Tabela 7 lista os cinco cenários.

Tabela 7: Cenários simulados e resultados.

#	Modo	Dific.	Veloc.	Situação	Pontos	s3
1	Siga	Fácil	Lento	Partida completa, 8 rodadas	36	8
2	Siga	Fácil	Rápido	Erro de propósito na rodada 3	6	8
3	Siga	Médio	Rápido	Partida completa, 16 rodadas	544	16
4	Siga	Difícil	Lento	Erro de propósito na rodada 2	3	32
5	Mando eu	Fácil	Lento	UP, RIGHT, DOWN; erro na rodada 3	3	8

Nos cinco cenários a pontuação foi igual à esperada, o `s3` bateu e as divergências na sequência de cores foram zero.

6.3 Resultados no Vivado

A Figura 4 mostra a execução completa no `xsim`, e a Figura 5 o resultado final no console do Vivado.

```
****** xsim v2024.1 (64-bit)
  **** SW Build 5076996 on Wed May 22 18:37:14 MDT 2024
  **** IP Build 5075265 on Wed May 22 21:45:21 MDT 2024
  **** SharedData Build 5076995 on Wed May 22 18:29:18 MDT 2024
  **** Start of session at: Tue Oct 13 08:56:35 2026
    ** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
    ** Copyright 2022-2024 Advanced Micro Devices, Inc. All Rights Reserved.

source xsim.dir/top_sim/xsim_script.tcl
# xsim {top_sim} -autoloadwcfg -runall
Time resolution is 1 ps
run -all
Genius no nucleo RISC-V (PBL03) - 161 palavras de programa carregadas

=== Cenario 1: modo Siga, facil, lento, partida completa (8 rodadas) ===
  pontuacao = 36 (esperado 36) | tamanho max s3 = 8 (esperado 8) | divergencias = 0
  >>> Cenario 1: modo Siga, facil, lento, partida completa (8 rodadas): PASSOU

=== Cenario 2: modo Siga, facil, rapido, erro de proposito na rodada 3 ===
  [821000 ns] LED 2 acende: LEFT/azul
  [834000 ns] jogador aperta LEFT/azul
  [834000 ns] LED 2 acende: LEFT/azul
  [847000 ns] LED 1 acende: DOWN/verde
  [859000 ns] jogador aperta LEFT/azul
  [860000 ns] jogador aperta DOWN/verde
  [860000 ns] LED 2 acende: LEFT/azul
  [873000 ns] LED 1 acende: DOWN/verde
  [885000 ns] LED 2 acende: LEFT/azul
  [897000 ns] jogador aperta RIGHT/amarelo
  [898000 ns] ECALL: jogo terminou, pontuacao impressa = 6
  pontuacao = 6 (esperado 6) | tamanho max s3 = 8 (esperado 8) | divergencias = 0
  >>> Cenario 2: modo Siga, facil, rapido, erro de proposito na rodada 3: PASSOU

=== Cenario 3: modo Siga, medio, rapido, partida completa (16 rodadas) ===
  pontuacao = 544 (esperado 544) | tamanho max s3 = 16 (esperado 16) | divergencias = 0
  >>> Cenario 3: modo Siga, medio, rapido, partida completa (16 rodadas): PASSOU

=== Cenario 4: modo Siga, dificil, lento, erro de proposito na rodada 2 ===
  pontuacao = 3 (esperado 3) | tamanho max s3 = 32 (esperado 32) | divergencias = 0
  >>> Cenario 4: modo Siga, dificil, lento, erro de proposito na rodada 2: PASSOU

=== Cenario 5: modo Mando eu, facil, lento, erro de proposito na rodada 3 ===
  [2708000 ns] jogador aperta UP/vermelho
  [2708000 ns] LED 0 acende: UP/vermelho
  [2730000 ns] jogador aperta UP/vermelho
  [2731000 ns] jogador aperta RIGHT/amarelo
  [2731000 ns] LED 0 acende: UP/vermelho
  [2754000 ns] LED 3 acende: RIGHT/amarelo
  [2776000 ns] jogador aperta UP/vermelho
  [2776000 ns] jogador aperta RIGHT/amarelo
  [2777000 ns] jogador aperta DOWN/verde
  [2777000 ns] LED 0 acende: UP/vermelho
  [2799000 ns] LED 3 acende: RIGHT/amarelo
  [2822000 ns] LED 1 acende: DOWN/verde
  [2844000 ns] jogador aperta DOWN/verde
  [2844000 ns] ECALL: jogo terminou, pontuacao impressa = 3
  pontuacao = 3 (esperado 3) | tamanho max s3 = 8 (esperado 8) | divergencias = 0
```

Figura 4: Execução no `xsim`: cada cenário termina com `PASSOU`.

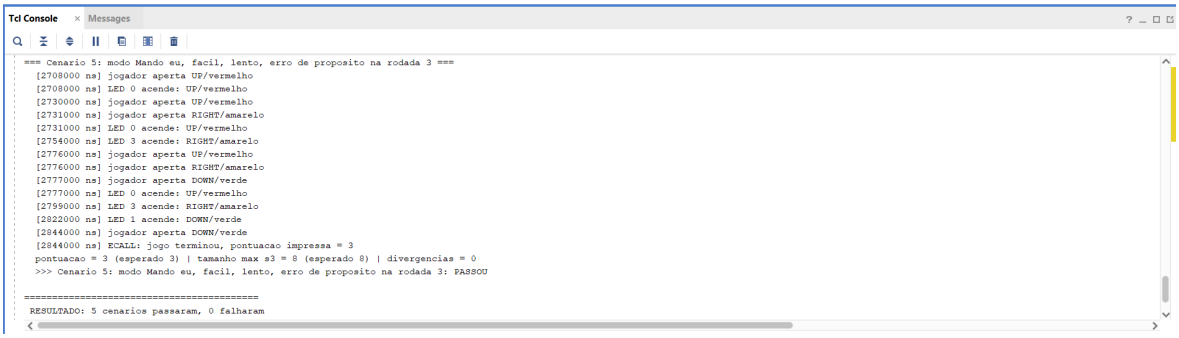


Figura 5: Console do Vivado: cenário 5 (Mando eu) com pontuação 3 e o resultado final `5 cenarios passaram, 0 falharam`.

A Figura 6 mostra as ondas da simulação inteira (0 a 2.845 µs). Cada pulso de `ecall` marca o fim de uma partida, e o `score` mostra 4294967295 (que representa -1) enquanto a partida está em andamento. O `led_prev` tem um bit por LED: 1 = UP, 2 = DOWN, 4 = LEFT, 8 = RIGHT.

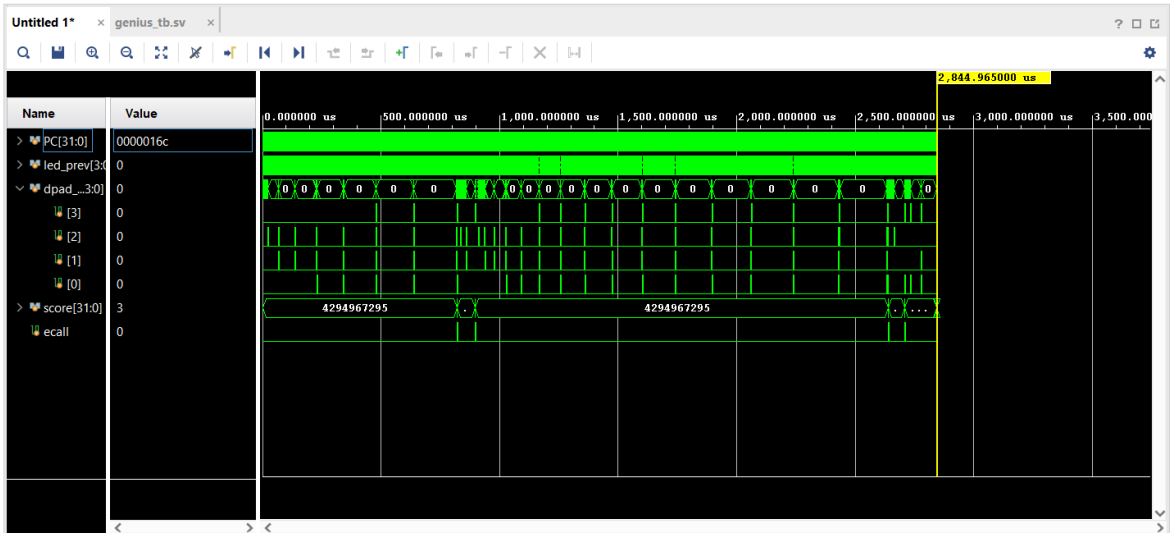


Figura 6: Ondas do Vivado na simulação inteira: `PC`, `led_prev`, `dpad_btn`, `score` e `ecall`.

Modo Siga. A Figura 7 mostra o cenário 2, de 800 a 900 µs. O jogo mostra a sequência crescendo no `led_prev` (4; 4, 2; 4, 2, 4), com o LED apagando entre as cores. O jogador erra de propósito na rodada 3 e a pontuação final é 6.

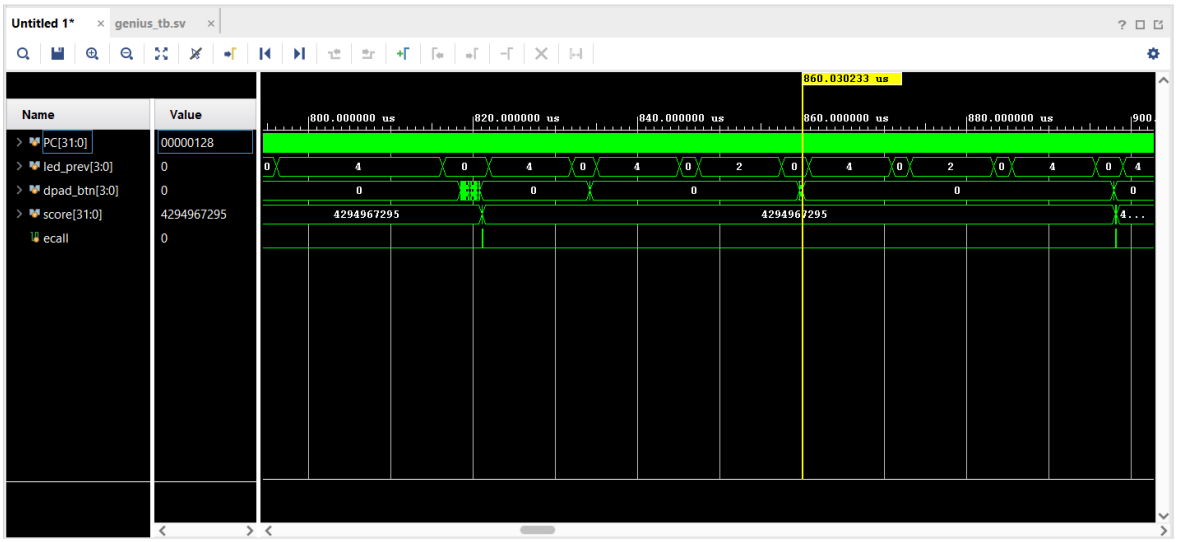


Figura 7: Modo Siga (cenário 2): a sequência de LEDs cresce a cada rodada.

Modo Mando eu. A Figura 8 mostra o cenário 5, de 2.700 a 2.850 µs. O jogador escolhe UP, RIGHT e DOWN, e o jogo repete a sequência acumulada (1; 1, 8; 1, 8, 2). A partida termina na rodada 3 com pontuação 3. O pulso de `ecall` em cerca de 2.708 µs marca o fim do cenário anterior.

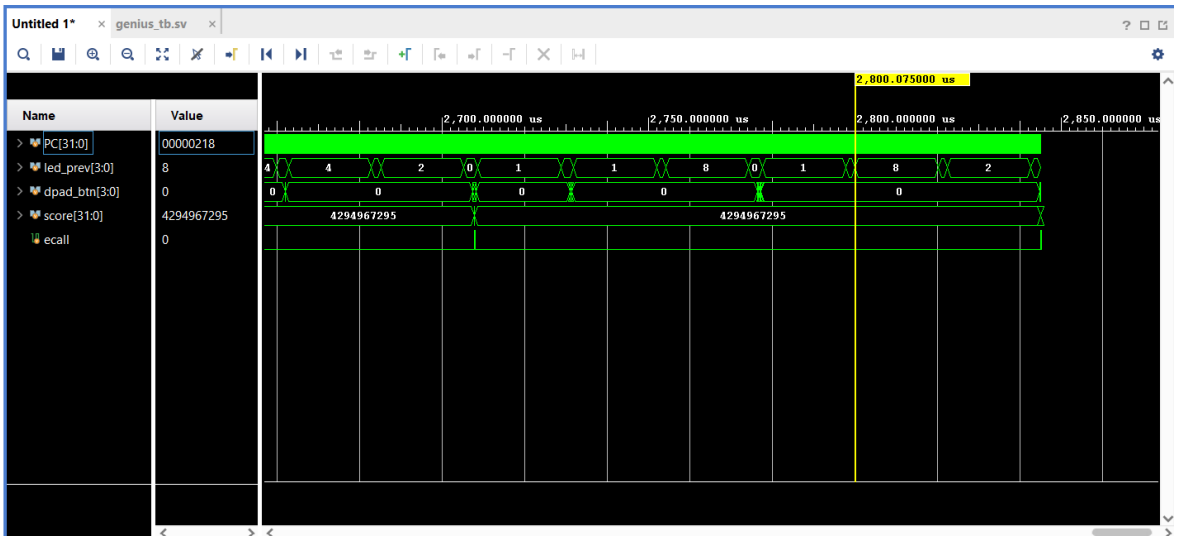


Figura 8: Modo Mando eu (cenário 5): apertos de botão e sequência repetida nos LEDs.

Processador em execução. Com zoom em nanossegundos (Figura 9), o `PC` muda a cada ciclo de clock (10 ns), mostrando o núcleo executando as instruções do Genius.

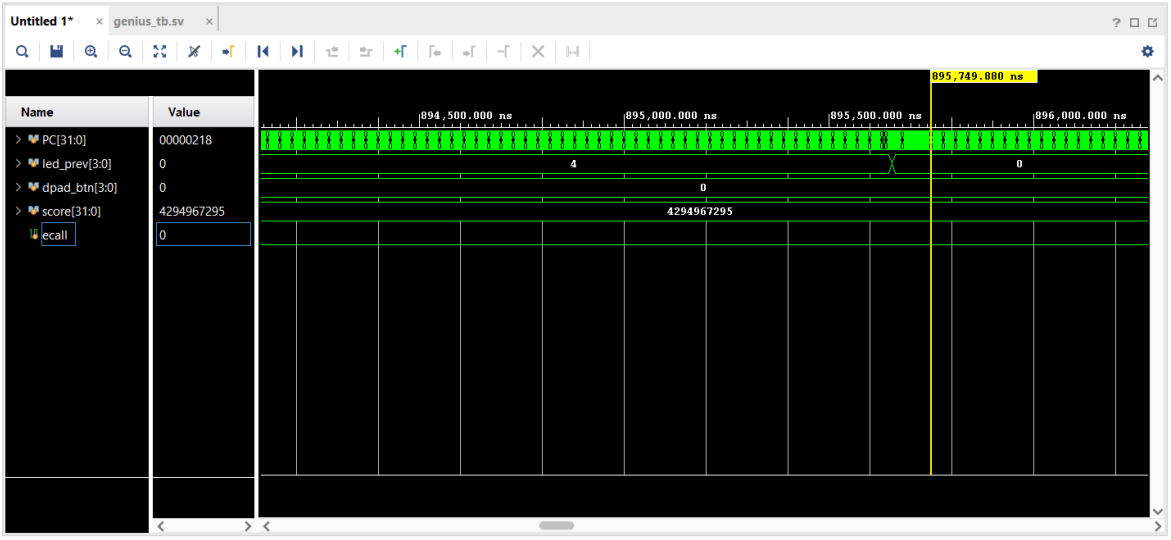


Figura 9: O `PC` muda a cada ciclo de clock na escala de nanossegundos (próximo de 895 µs).

7 Diagramas gerados pelo Vivado

O *elaborated design* do Vivado desenha o circuito a partir do próprio RTL. Ele serve como prova de que o código corresponde à microarquitetura do Capítulo 7: o `riscvsingle` aparece dividido em controller e datapath (Figura 1), como no livro.

O SoC de teste é o módulo `genius_soc` (Figura 10). Nele, `rv` é o processador, `imem1` guarda o programa Genius, `dmem1` é a memória de dados e `pp` são os periféricos do jogo. As entradas externas são `clk`, `reset`, `switches[4:0]` e `dpad_btn[3:0]`; a saída é `leds[95:0]`. O comparador `RTL_EQ` detecta endereços que começam em `0xF` (periféricos), a porta `RTL_AND` impede que essas escritas cheguem à `dmem1`, e o `RTL_MUX` escolhe se o dado lido vem da memória ou dos periféricos.

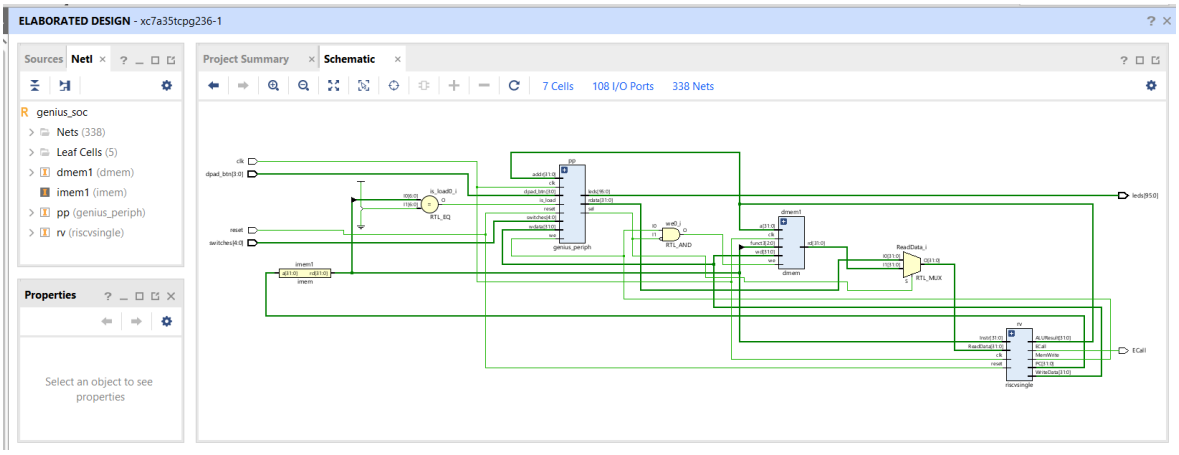


Figura 10: `genius_soc`: processador (`rv`), memórias (`imem1`, `dmem1`) e periféricos do Genius (`pp`).

A Figura 11 mostra o módulo `rv` aberto dentro do SoC, com o controller (`c`) e o

datapath (`dp`) ligados pelos sinais de controle.

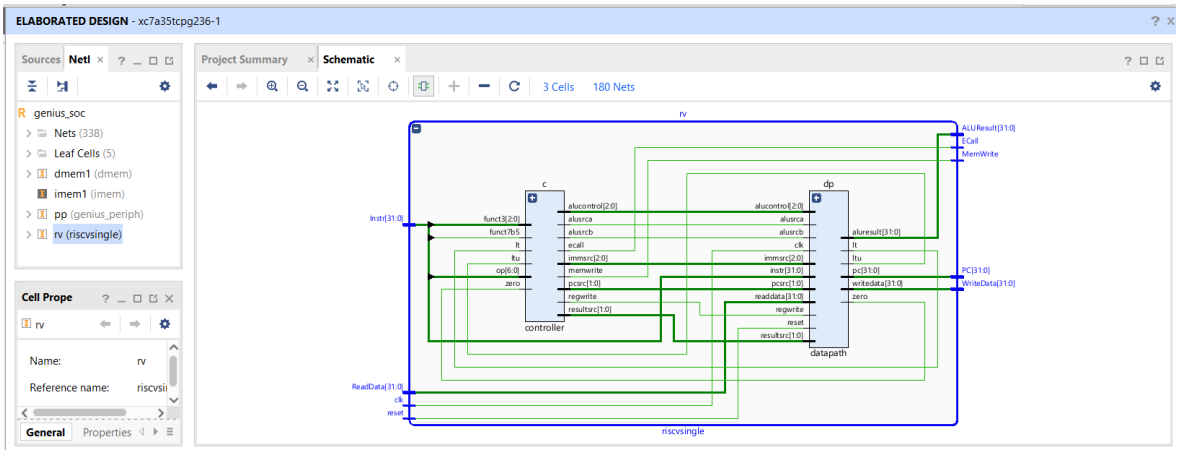


Figura 11: O módulo `rv` (`riscvsingle`) aberto dentro do SoC.

7.1 Como os diagramas foram obtidos

1. No Vivado, criar um projeto só para visualização e adicionar os fontes: `add_files [glob .../pbl03/rtl/*.sv]`.
2. Definir o módulo de topo: `set_property top riscvsingle [current_fileset]` para o processador, ou `genius_soc` para o sistema.
3. Para o `genius_soc`, aumentar o limite de elaboração das memórias (comando no Apêndice A).
4. Em *RTL ANALYSIS*, abrir *Open Elaborated Design* e depois *Schematic*. Um duplo clique em um bloco abre o nível de dentro.

O aviso do Vivado sobre não achar `genius.hex` e `data.hex` nessa etapa não afeta o diagrama; só afeta o conteúdo inicial das memórias.

8 Conclusão e próximos passos

O processador de ciclo único do Capítulo 7 foi implementado em SystemVerilog, estendido para 22 instruções e validado em simulação. O teste de sanidade passou, e o Genius do PBL02 rodou sem alteração nos cinco cenários, com pontuação e sequência de cores iguais às esperadas. Os diagramas do Vivado mostram a mesma divisão em controller e datapath do livro.

8.1 Limitações

- A verificação é somente em simulação. LEDs, chaves, botões e relógio do Genius são modelos em RTL; não houve síntese, implementação ou teste na placa.

- O relógio do modelo avança 1 ms por ciclo de clock, para que os atrasos de 2 s do jogo custem cerca de 2000 ciclos. Os tempos em microssegundos das ondas, portanto, não são tempos reais de jogo.
- O testbench verifica a sequência de cores e a pontuação em cinco cenários escolhidos; não é uma verificação exaustiva das instruções.

8.2 Próximos passos

- **Frente 2 do PBL03:** montar o ambiente de verificação em UVM a partir do `verif-env-template` [2] e verificar o processador da outra equipe. Esta etapa ainda não foi iniciada.
- Aplicar o mesmo ambiente ao processador desta equipe como alvo de teste, antes de trocar para o da outra equipe.
- Integrar os resultados da verificação (cobertura e *scoreboard*) a este relatório e à apresentação de 10 minutos com demonstração.

Referências

[1] HARRIS, S. L.; HARRIS, D. M. *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann, 2022. Capítulo 7: Microarquitetura.

[2] NELSONAFN. *verif-env-template*. Repositório GitHub. Disponível em: `https://github.com/nelsonafn/verif-env-template`. Estrutura de projeto usada no PBL03.

[3] RISC-V INTERNATIONAL. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. Disponível em: `https://riscv.org/technical/specifications/`.

[4] AMD. *Vivado Design Suite User Guide: Logic Simulation (UG900)*, versão 2024.1.

A Como reproduzir

Ambiente: Windows, Git Bash (MINGW64) e Vivado 2024.1 instalado em `C:\Xilinx\Vivado\2024.1`. Os comandos são executados na pasta do projeto (`pbl03`), um por vez.

Teste de sanidade do processador

```
source setup_env.sh
rm -rf build
cmake -S . -B build -G "Unix Makefiles" -DTOP_NAME=testbench -DTEST_NAME=sanity -
    DSEED=1 -DVIVADO_PARMS="--R"
make SHELL="$(which bash)" compile
make SHELL="$(which bash)" sim
cd build
cmd //c "xsim top_sim -R"
```

Genius no processador

```
source setup_env.sh
rm -rf build
cmake -S . -B build -G "Unix Makefiles" -DTOP_NAME=genius_tb -DTEST_NAME=sanity -
    DSEED=1 -DVIVADO_PARMS="--R"
cp genius.hex data.hex build/
make SHELL="$(which bash)" compile
make SHELL="$(which bash)" sim
cd build
cmd //c "xsim top_sim -R"
```

O resultado esperado é a linha `RESULTADO: 5 cenarios passaram, 0 falharam`. Para ver as ondas, o último comando passa a ser `cmd //c "xsim top_sim -gui"`; no console Tcl do Vivado, adicionam-se os sinais e roda-se a simulação:

```
add_wave -radix hex /genius_tb/dut/PC
add_wave /genius_tb/led_prev
add_wave /genius_tb/dpad_btn
add_wave -radix unsigned /genius_tb/score
add_wave /genius_tb/ecall
run all
```

Diagramas (elaborated design)

No console Tcl de um Vivado aberto, um comando por vez:

```
create_project pbl03_view ../pbl03_view -part xc7a35tcpg236-1 -force
add_files [glob ../pbl03/rtl/*.sv]
set_property top genius_soc [current_fileset]
set_param synth.elaboration.rodinMoreOptions {rt::set_parameter
    dissolveMemorySizeLimit 262144}
```

Depois, em *RTL ANALYSIS*, abrir *Open Elaborated Design* e *Schematic*. Para ver

só o processador, usar `riscvsingle` como topo.

Gerar o programa Genius

O `genius.hex` e o `data.hex` são gerados pelo script `genius/build_genius.sh` (Apêndice B), que usa o montador e o ligador do RISC-V GNU Toolchain; os arquivos prontos já acompanham o projeto.

Observações do ambiente

O último `make sim` termina com o erro `Unsupported architecture: i686-pc`, do script de inicialização do Xilinx no Git Bash. A elaboração já foi concluída e o `xsim` é chamado à parte, por `cmd //c`. No Git Bash, colar várias linhas de uma vez pode inserir caracteres extras; os comandos funcionam digitados ou colados uma linha por vez.

B Código do SoC de teste do Genius

Este apêndice reúne os arquivos novos criados para executar o Genius no processador. O restante do RTL (controller, datapath, ALU e demais módulos) está descrito na Seção 3.

`rtl/genius_soc.sv`

```
// ============================================================
// genius_soc.sv  SoC de simulacao para rodar o Genius no nucleo
//
//   riscvsingle (nucleo)  +  imem (genius.hex)  +  dmem (data.hex)
//   + genius_periph (LEDs, switches, D-Pad, RTC)
//
// Decodificacao de endereco: 0xF000_0xxx = periferico, resto = dmem.
// ============================================================
module genius_soc(
    input  logic        clk, reset,
    input  logic [4:0]  switches,
    input  logic [3:0]  dpad_btn,
    output logic [95:0] leds,
    output logic        ECall
);
    logic [31:0] PC, Instr, ReadData, ALUResult, WriteData;
    logic        MemWrite;

    logic        io_sel;
    logic [31:0] io_rdata, mem_rdata;

    riscvsingle rv(
        .clk(clk), .reset(reset),
        .PC(PC), .Instr(Instr),
```

```
        .MemWrite(MemWrite),
        .ALUResult(ALUResult), .WriteData(WriteData),
        .ReadData(ReadData),
        .ECall(ECall)
    );

    imem #(.PROG_FILE("genius.hex")) imem1(.a(PC), .rd(Instr));

    dmem #(.DATA_FILE("data.hex")) dmem1(
        .clk(clk), .we(MemWrite & ~io_sel), .funct3(Instr[14:12]),
        .a(ALUResult), .wd(WriteData), .rd(mem_rdata)
    );

    genius_periph pp(
        .clk(clk), .reset(reset),
        .we(MemWrite),
        .is_load(Instr[6:0] == 7'b0000011),
        .addr(ALUResult), .wdata(WriteData),
        .sel(io_sel), .rdata(io_rdata),
        .switches(switches), .dpad_btn(dpad_btn), .leds(leds)
    );

    assign ReadData = io_sel ? io_rdata : mem_rdata;
endmodule
```

rtl/genius_periph.sv

```
// ============================================================
// genius_periph.sv  Perifericos mapeados em memoria (modelo de simulacao)
//
// Reproduz, para o nosso nucleo, os 4 perifericos que o Ripes oferecia
// ao programa Genius do PBL02. Regiao de I/O: enderecos 0xF000_0xxx.
//
//   0xF0000000 + 4*i  LED Matrix   (i=0..3)  escrita: cor RGB de 24 bits
//   0xF0000100        Switches                leitura: bits [4:0]
//   0xF0000200 + 4*d  D-Pad (d=0 UP,1 DOWN,2 LEFT,3 RIGHT)
//                                             leitura: 1 se houve clique
//                                             desde a ultima leitura
//                                             (captura de borda; a leitura limpa)
//   0xF0000300        RTC                     leitura: tempo em "ns"
//
// RTC_STEP: quanto o RTC avanca por ciclo de clock. Em hardware real o
// RTC avanca em tempo real; em simulacao usamos 1.000.000 ns/ciclo
// (1 ms por clock) para que um delay de 2 s custe so ~2000 ciclos.
// ============================================================
module genius_periph #(
    parameter RTC_STEP = 1000000
)(
    input  logic        clk, reset,
    // interface com o barramento de dados do processador
    input  logic        we,        // MemWrite
    input  logic        is_load,   // instrucao corrente e um load
    input  logic [31:0] addr,      // ALUResult
    input  logic [31:0] wdata,
    output logic        sel,       // endereco cai na regiao de I/O
    output logic [31:0] rdata,
```

```
    // "mundo externo" (controlado pelo testbench)
    input  logic [4:0]  switches,
    input  logic [3:0]  dpad_btn,  // pulso de 1 ciclo = clique
    output logic [95:0] leds       // 4 LEDs x 24 bits (LED i = leds[24*i +: 24])
);
    logic [3:0]  latch;
    logic [31:0] rtc;

    assign sel = (addr[31:28] == 4'hF);

    wire [3:0] page = addr[11:8];
    wire [1:0] idx  = addr[3:2];

    // leitura combinacional
    always_comb begin
        rdata = 32'b0;
        case (page)
            4'h1:    rdata = {27'b0, switches};
            4'h2:    rdata = {31'b0, latch[idx]};
            4'h3:    rdata = rtc;
            default: rdata = 32'b0;
        endcase
    end

    always_ff @(posedge clk) begin
        if (reset) begin
            latch <= 4'b0;
            rtc   <= 32'b0;
            leds  <= 96'b0;
        end else begin
            rtc <= rtc + RTC_STEP;

            // escrita nos LEDs
            if (we && sel && page == 4'h0)
                leds[24*idx +: 24] <= wdata[23:0];

            // D-Pad: o clique seta o latch; a leitura (lw) limpa
            for (int i = 0; i < 4; i++) begin
                if (dpad_btn[i])
                    latch[i] <= 1'b1;
                else if (is_load && sel && page == 4'h2 && idx == i)
                    latch[i] <= 1'b0;
            end
        end
    end
endmodule
```

genius/crt0.s

```
# crt0.s  inicializacao minima (o Ripes faz isso sozinho; no nosso SoC
# precisamos setar o ponteiro de pilha antes de entrar no main).
# sp = 0x3FF0 (topo da memoria de dados de 16 KB, alinhado)
.text
.globl _start
_start:
    lui  sp, 0x4        # sp = 0x00004000
```

```
    addi sp, sp, -16    # sp = 0x00003FF0
# (cai direto no main, que e a primeira instrucao do genius_pbl02.s)
```

genius/build_genius.sh

```
#!/bin/bash
# Monta o Genius para o nosso nucleo (so precisa rodar se alterar o .s)
set -e
AS="riscv64-unknown-elf-as -march=rv32i -mabi=ilp32 -mno-relax"
SYM="--defsym LED_MATRIX_0_BASE=0xF0000000 --defsym SWITCHES_0_BASE=0xF0000100 --defsym D_PAD_0_UP=0
     xF0000200 --defsym RTC_0_BASE=0xF0000300"
$AS crt0.s -o crt0.o
$AS $SYM genius_pbl02.s -o genius.o
riscv64-unknown-elf-ld -m elf32lriscv -Ttext=0x0 -Tdata=0x2000 -o genius.elf crt0.o genius.o
riscv64-unknown-elf-objcopy -O binary -j .text genius.elf text.bin
riscv64-unknown-elf-objcopy -O binary -j .data genius.elf data.bin
python3 -I make_hex.py
riscv64-unknown-elf-objdump -d -M no-aliases genius.elf > genius.dis
riscv64-unknown-elf-nm genius.elf | sort > genius.sym
```